

Actividad 2: Programación Orientada a Objetos

SANTIAGO ALVAREZ NARANJO

Santiago Alvarez Naranjo

Docente: Walter Hugo Arboleda Mazo | Universidad Nacional de Colombia Sede Medellín

Contenido

Punto 1.....	2
Enunciado: Ejercicio 2.1 página 63.....	2
Ejercicios propuestos.	2
Código	2
Diagrama de clases.....	3
Punto 2.....	3
Enunciado: Ejercicio 2.2 página 66.....	3
Ejercicios propuestos.	4
Código	4
Diagrama de clases.....	6
Punto 3.....	6
Enunciado: Ejercicio 2.3 página 66.....	6
Ejercicios propuestos.	7
Código.	7
Diagrama de clases.....	10
Punto 4.....	10
Enunciado: Ejercicio 2.4 página 86.....	10
Ejercicios propuestos.	11
Código.	11
Diagrama de clases.....	13
Punto 5.....	14
Enunciado: Ejercicio 2.5 página 95.....	14
Ejercicios propuestos.	14
Código.	14
Diagrama de clases.....	16
URL Github:	16

Punto 1.

Enunciado: Ejercicio 2.1 página 63.

Se requiere un programa que modele el concepto de una persona. Una persona posee nombre, apellido, número de documento de identidad y año de nacimiento. La clase debe tener un constructor que inicialice los valores de sus respectivos atributos. La clase debe incluir los siguientes métodos:

- Definir un método que imprima en pantalla los valores de los atributos del objeto.
- En un método main se deben crear dos personas y mostrar los valores de sus atributos en pantalla.

Ejercicios propuestos.

- Agregar dos nuevos atributos a la clase Persona. Un atributo que represente el país de nacimiento de la persona (de tipo String) y otro que identifique el género de la persona, el cual debe representarse como un char con valores 'H' o 'M'.
- Modificar el constructor de la clase Persona para que inicialice estos dos nuevos atributos.
- u Modificar el método imprimir de la clase Persona para que muestre en pantalla los valores de los nuevos atributos.

Código

```
## Punto 1: Ejercicio 2.1 página 63
class Persona:
    def __init__(self, nombre, apellido, documento, nacimiento, pais,
genero):
        self.nombre = nombre
        self.apellido = apellido
        self.documento = documento
        self.nacimiento = nacimiento
        self.pais = pais
        self.genero = genero
    def mostrar_datos(self):
        print(f"Nombre: {self.nombre}")
        print(f"Apellido: {self.apellido}")
        print(f"Documento: {self.documento}")
        print(f"Nacimiento: {self.nacimiento}")
        print(f"País: {self.pais}")
        print(f"Género: {self.genero}")
personal = Persona("Santiago", "Alvarez", "1023633994", 2008,
"Colombia", "H")
persona2 = Persona("Sara", "Vásquez", "1023040708", 2008, "Alemania",
"M")
print("--- Datos de la Persona 1 ---")
personal.mostrar_datos()
print("--- Datos de la Persona 2 ---")
persona2.mostrar_datos()
```

Diagrama de clases.

Persona
+ nombre : String + apellido : String + documento : String + nacimiento : Integer + pais : String + genero : String
+ __init__(nombre : String, apellido : String, documento : String, nacimiento : Integer, pais : String, genero : String) : void + mostrar_datos() : void

Punto 2.

Enunciado: Ejercicio 2.2 página 66.

Se requiere un programa que modele el concepto de un planeta del sistema solar. Un planeta tiene los siguientes atributos: u Un nombre de tipo String con valor inicial de null.

- Cantidad de satélites de tipo int con valor inicial de cero.
- Masa en kilogramos de tipo double con valor inicial de cero.
- Volumen en kilómetros cúbicos de tipo double con valor inicial de cero.
- Diámetro en kilómetros de tipo int con valor inicial de cero.
- Distancia media al Sol en millones de kilómetros, de tipo int con valor inicial de cero.
- Tipo de planeta de acuerdo con su tamaño, de tipo enumerado con los siguientes valores posibles: GASEOSO, TERRESTRE y ENANO.
- Observable a simple vista, de tipo booleano con valor inicial false.

La clase debe incluir los siguientes métodos:

- La clase debe tener un constructor que inicialice los valores de sus respectivos atributos.
- Definir un método que imprima en pantalla los valores de los atributos de un planeta.
- Calcular la densidad un planeta, como el cociente entre su masa y su volumen.
- Determinar si un planeta del sistema solar se considera exterior. Un planeta exterior está situado más allá del cinturón de asteroides. El cinturón de asteroides se encuentra entre 2.1 y 3.4 UA. Una unidad astronómica (UA) es la distancia entre la Tierra y el Sol= 149597870 Km.
- En un método main se deben crear dos planetas y mostrar los valores de sus atributos en pantalla. Además, se debe imprimir la densidad de cada planeta y si el planeta es un planeta exterior del sistema solar.



Ejercicios propuestos.

- Agregar dos atributos a la clase Planeta. El primero debe representar el periodo orbital del planeta (en años). El segundo atributo representa el periodo de rotación (en días).
- Modificar el constructor de la clase para que inicialice los valores de estos dos nuevos atributos.
- Modificar el método imprimir para que muestre en pantalla los valores de los nuevos atributos.

Código

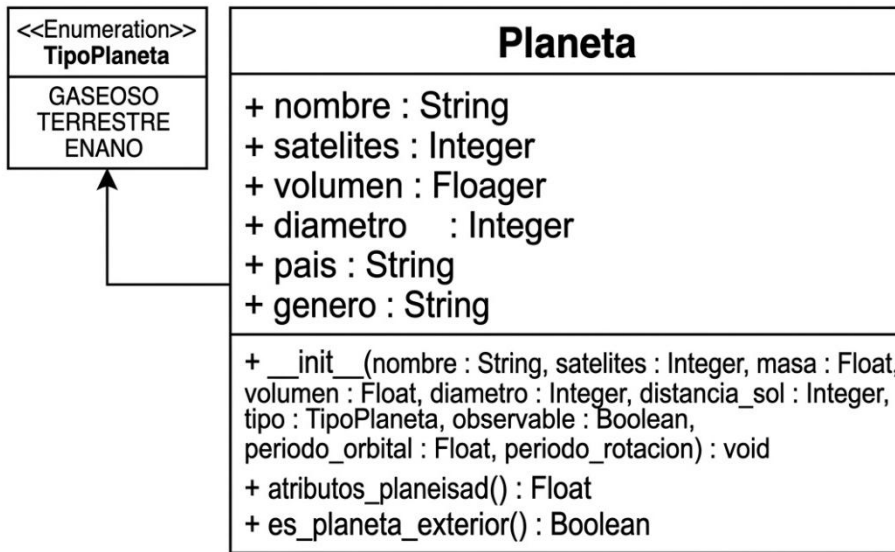
```
## Punto 2: Ejercicio 2.2 página 66
from enum import Enum
class TipoPlaneta(Enum):
    GASEOSO = "GASEOSO"
    TERRESTRE = "TERRESTRE"
    ENANO = "ENANO"
class Planeta:
    def __init__(self, nombre=None, satelites=0, masa=0.0, volumen=0.0, diametro=0,
                  distancia_sol=0, tipo=None, observable=False, periodo_orbital=0.0, periodo_rotacion=0.0):
        self.nombre = nombre
        self.satelites = satelites
        self.masa = masa
        self.volumen = volumen
        self.diametro = diametro
        self.distancia_sol = distancia_sol
        self.tipo = tipo
        self.observable = observable
        self.periodo_orbital = periodo_orbital
        self.periodo_rotacion = periodo_rotacion
    def atributos_planeta(self):
        print(f"Nombre: {self.nombre}")
        print(f"Satélites: {self.satelites}")
        print(f"Masa: {self.masa} kg")
        print(f"Volumen: {self.volumen} km3")
        print(f"Diámetro: {self.diametro} km")
        print(f"Distancia al Sol: {self.distancia_sol} millones de km")
        print(f"Tipo: {self.tipo.value if self.tipo else None}")
        print(f"Observable a simple vista: {self.observable}")
        print(f"Periodo Orbital: {self.periodo_orbital} años")
        print(f"Periodo de Rotación: {self.periodo_rotacion} días")
    def calcular_densidad(self):
        if self.volumen == 0:
            return 0.0
        densidad = self.masa / self.volumen
        print(f"La densidad de {self.nombre} es: {densidad} kg/km3")
        return densidad
```



```
def es_planeta_exterior(self):
    limite_exterior_millones = 3.4 * 149.597870
    if self.distancia_sol > limite_exterior_millones:
        print(f"El planeta {self.nombre} ES un planeta exterior.")
        return True
    else:
        print(f"El planeta {self.nombre} NO es un planeta exterior.")
        return False

planeta1 = Planeta(
    nombre="Kamikase",
    satelites=23,
    masa=5.97e24,
    volumen=1.08e12,
    diametro=12742,
    distancia_sol=600,
    tipo=TipoPlaneta.GASEOSO,
    observable=True,
    periodo_orbital=12.5,
    periodo_rotacion=0.9
)
planeta1.atributos_planeta()
planeta1.calcular_densidad()
planeta1.es_planeta_exterior()
print("---" * 30)
planeta2 = Planeta(
    nombre="Renacuajo",
    satelites=2,
    masa=2.0e12,
    volumen=1232898.0,
    diametro=140,
    distancia_sol=150,
    tipo=TipoPlaneta.TERRESTRE,
    observable=False,
    periodo_orbital=1.0,
    periodo_rotacion=24.3
)
planeta2.atributos_planeta()
planeta2.calcular_densidad()
planeta2.es_planeta_exterior()
```

Diagrama de clases.



Punto 3.

Enunciado: Ejercicio 2.3 página 66.

Se requiere un programa que modele el concepto de un automóvil. Un automóvil tiene los siguientes atributos: u Marca: el nombre del fabricante.

- Modelo: año de fabricación.
- Motor: volumen en litros del cilindraje del motor de un automóvil.
- Tipo de combustible: valor enumerado con los posibles valores de gasolina, bioetanol, diésel, biodiésel, gas natural.
- Tipo de automóvil: valor enumerado con los posibles valores de carro de ciudad, subcompacto, compacto, familiar, ejecutivo, SUV.
- Número de puertas: cantidad de puertas.
- Cantidad de asientos: número de asientos disponibles que tiene el vehículo.
- Velocidad máxima: velocidad máxima sostenida por el vehículo en km/h.
- Color: valor enumerado con los posibles valores de blanco, negro, rojo, naranja, amarillo, verde, azul, violeta.
- Velocidad actual: velocidad del vehículo en un momento dado.
- La clase debe incluir los siguientes métodos:
- Un constructor para la clase Automóvil donde se le pasen como parámetros los valores de sus atributos.
- Métodos get y set para la clase Automóvil.
- Métodos para acelerar una cierta velocidad, desacelerar y frenar (colocar la velocidad actual en cero). Es importante tener en cuenta que no se debe acelerar más allá de la velocidad máxima permitida para el automóvil. De igual manera, tampoco es posible desacelerar a una velocidad negativa. Si se cumplen estos casos, se debe mostrar por pantalla los mensajes correspondientes.



- Un método para calcular el tiempo estimado de llegada, utilizando como parámetro la distancia a recorrer en kilómetros. El tiempo estimado se calcula como el cociente entre la distancia a recorrer y la velocidad actual.
- Un método para mostrar los valores de los atributos de un Automóvil en pantalla.
- Un método main donde se deben crear un automóvil, colocar su velocidad actual en 100 km/h, aumentar su velocidad en 20 km/h, luego decrementar su velocidad en 50 km/h, y después frenar. Con cada cambio de velocidad, se debe mostrar en pantalla la velocidad actual.

Ejercicios propuestos.

- Agregar a la clase Automóvil, un atributo para determinar si el vehículo es automático o no. Agregar los métodos get y set para dicho atributo. Modificar el constructor para inicializar dicho atributo.
- Modificar el método acelerar para que si la velocidad máxima se sobrepase se genere una multa. Dicha multa se puede incrementar cada vez que el vehículo intenta superar la velocidad máxima permitida.
- Agregar un método para determinar si un vehículo tiene multas y otro método para determinar el valor total de multas de un vehículo.

Código.

```
from enum import Enum

class TipoCombustible(Enum):
    GASOLINA = "Gasolina"
    BIOETANOL = "Bioetanol"
    DIESEL = "Diésel"
    BIODIESEL = "Biodiésel"
    GAS_NATURAL = "Gas Natural"

class TipoAutomovil(Enum):
    CARRO_CIUDAD = "Carro de ciudad"
    SUBCOMPACTO = "Subcompacto"
    COMPACTO = "Compacto"
    FAMILIAR = "Familiar"
    EJECUTIVO = "Ejecutivo"
    SUV = "SUV"

class Color(Enum):
    BLANCO = "Blanco"
    NEGRO = "Negro"
    ROJO = "Rojo"
    NARANJA = "Naranja"
    AMARILLO = "Amarillo"
    VERDE = "Verde"
    AZUL = "Azul"
    VIOLETA = "Violeta"

class Automovil:
    def __init__(self, marca, modelo, motor, tipo_combustible, tipo_automovil,
```




```
numero_puertas, cantidad_asientos, velocidad_maxima, color, velocidad_actual=0,
es_automatico=False):
    self.marca = marca
    self.modelo = modelo
    self.motor = motor
    self.tipo_combustible = tipo_combustible
    self.tipo_automovil = tipo_automovil
    self.numero_puertas = numero_puertas
    self.cantidad_asientos = cantidad_asientos
    self.velocidad_maxima = velocidad_maxima
    self.color = color
    self.velocidad_actual = velocidad_actual
    self.es_automatico = es_automatico
    self.multas = 0
    self.valor_multa = 150000
def set_velocidad_actual(self, velocidad_actual):
    if velocidad_actual < 0:
        print("La velocidad no puede ser negativa.")
    elif velocidad_actual > self.velocidad_maxima:
        print("No se puede superar la velocidad máxima permitida.")
    else:
        self.velocidad_actual = velocidad_actual
def acelerar(self, incremento):
    if self.velocidad_actual + incremento > self.velocidad_maxima:
        print(
            f"¡Alerta! Intentó superar la velocidad máxima permitida ({self.velocidad_maxima} km/h)."
        )
        self.multas += 1
        print(
            f"Se ha generado una multa. Total de multas acumuladas: {self.multas}"
        )
        self.velocidad_actual = self.velocidad_maxima
    else:
        self.velocidad_actual += incremento
def desacelerar(self, decremento):
    if self.velocidad_actual - decremento < 0:
        print(
            "No es posible desacelerar a una velocidad negativa. El vehículo se detuvo."
        )
        self.velocidad_actual = 0
    else:
        self.velocidad_actual -= decremento
def frenar(self):
    self.velocidad_actual = 0
def calcular_tiempo_llegada(self, distancia):
```



```
if self.velocidad_actual == 0:
    print(
        "El vehículo está detenido. No se puede calcular el tiempo estimado de llegada."
    )
    return None
return distancia / self.velocidad_actual

def tiene_multas(self):
    return self.multas > 0

def valor_total_multas(self):
    return self.multas * self.valor_multa

def mostrar_datos(self):
    print(f"Marca: {self.marca}")
    print(f"Modelo: {self.modelo}")
    print(f"Motor: {self.motor} L")
    print(
        f"Tipo de Combustible: {self.tipo_combustible.value if self.tipo_combustible else None}"
    )
    print(
        f"Tipo de Automóvil: {self.tipo_automovil.value if self.tipo_automovil else None}"
    )
    print(f"Número de Puertas: {self.numero_puertas}")
    print(f"Cantidad de Asientos: {self.cantidad_asientos}")
    print(f"Velocidad Máxima: {self.velocidad_maxima} km/h")
    print(f"Color: {self.color.value if self.color else None}")
    print(f"Velocidad Actual: {self.velocidad_actual} km/h")
    print(f"Es Automático: {self.es_automatico}")
    print(f"Tiene Multas: {self.tiene_multas()}")
    print(f"Total a pagar en Multas: ${self.valor_total_multas()}")

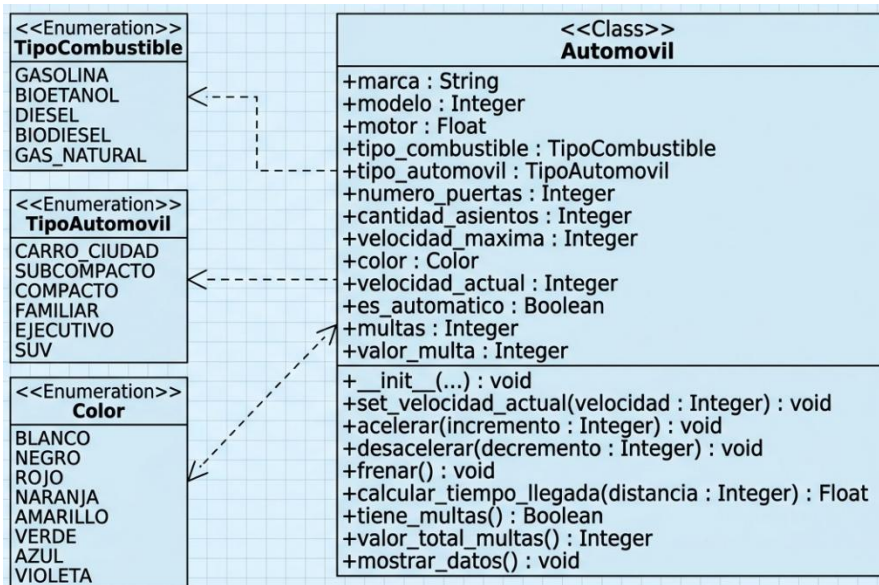
auto1 = Automovil(
    marca="Toyota",
    modelo=2022,
    motor=2.0,
    tipo_combustible=TipoCombustible.GASOLINA,
    tipo_automovil=TipoAutomovil.COMPACTO,
    numero_puertas=4,
    cantidad_asientos=5,
    velocidad_maxima=180,
    color=Color.ROJO,
    velocidad_actual=0,
    es_automatico=True
)

print("--- Datos iniciales del Automóvil ---")
auto1.mostrar_datos()
print("\n--- Pruebas de velocidad ---")
```



```
auto1.set_velocidad_actual(100)
print(f"Velocidad actual: {auto1.velocidad_actual} km/h")
auto1.acelerar(20)
print(f"Velocidad actual tras acelerar 20 km/h: {auto1.velocidad_actual} km/h")
auto1.desacelerar(50)
print(f"Velocidad actual tras desacelerar 50 km/h: {auto1.velocidad_actual} km/h")
distancia = 250
tiempo = auto1.calcular_tiempo_llegada(distancia)
if tiempo:
    print(f"Tiempo estimado para recorrer {distancia} km: {tiempo:.2f} horas")
print("\n--- Prueba de exceso de velocidad ---")
auto1.acelerar(120)
print(f"Velocidad actual tras el intento: {auto1.velocidad_actual} km/h")
auto1.frenar()
print(f"Velocidad actual tras frenar: {auto1.velocidad_actual} km/h")
print("\n--- Estado final del Automóvil ---")
auto1.mostrar_datos()
```

Diagrama de clases.



Punto 4.

Enunciado: Ejercicio 2.4 página 86.

Se requiere un programa que modele varias figuras geométricas: el círculo, el rectángulo, el cuadrado y el triángulo rectángulo.

- El círculo tiene como atributo su radio en centímetros.



- El rectángulo, su base y altura en centímetros. u El cuadrado, la longitud de sus lados en centímetros.
- El triángulo, su base y altura en centímetros. Se requieren métodos para determinar el área y el perímetro de cada figura geométrica. Además, para el triángulo rectángulo se requiere:
 - Un método que calcule la hipotenusa del rectángulo.
 - Un método para determinar qué tipo de triángulo es:
 - Equilátero: todos sus lados son iguales.
 - Isósceles: tiene dos lados iguales.
 - Escaleno: todos sus lados son diferentes

Ejercicios propuestos.

- Agregar una nueva clase denominada Rombo. Definir los métodos para calcular el área y el perímetro de esta nueva figura geométrica.
- Agregar una nueva clase denominada Trapecio. Definir los métodos para calcular el área y el perímetro de esta nueva figura geométrica.

Código.

```
import math
class Circulo:
    def __init__(self, radio):
        self.radio = radio
    def calcular_area(self):
        return math.pi * (self.radio**2)
    def calcular_perimetro(self):
        return 2 * math.pi * self.radio
class Rectangulo:
    def __init__(self, base, altura):
        self.base = base
        self.altura = altura
    def calcular_area(self):
        return self.base * self.altura
    def calcular_perimetro(self):
        return 2 * (self.base + self.altura)
class Cuadrado:
    def __init__(self, lado):
        self.lado = lado
    def calcular_area(self):
        return self.lado**2
    def calcular_perimetro(self):
        return 4 * self.lado
class TrianguloRectangulo:
    def __init__(self, base, altura):
        self.base = base
        self.altura = altura
```

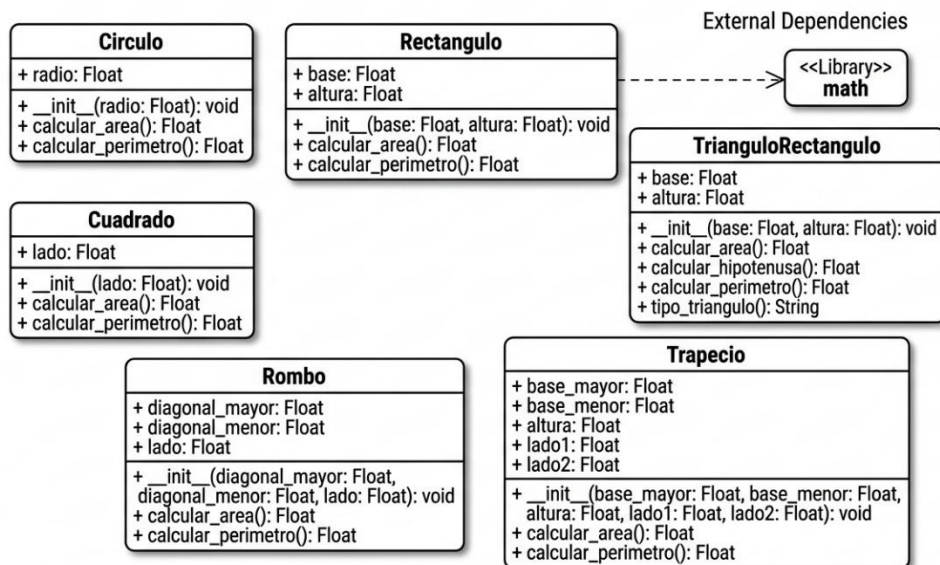


```
def calcular_area(self):
    return (self.base * self.altura) / 2
def calcular_hipotenusa(self):
    return math.sqrt(self.base**2 + self.altura**2)
def calcular_perimetro(self):
    return self.base + self.altura + self.calcular_hipotenusa()
def tipo_triangulo(self):
    hipotenusa = self.calcular_hipotenusa()
    if self.base == self.altura and self.altura == hipotenusa:
        return "Equilátero"
    elif (
        self.base == self.altura
        or self.base == hipotenusa
        or self.altura == hipotenusa
    ):
        return "Isósceles"
    else:
        return "Escaleno"
class Rombo:
    def __init__(self, diagonal_mayor, diagonal_menor, lado):
        self.diagonal_mayor = diagonal_mayor
        self.diagonal_menor = diagonal_menor
        self.lado = lado
    def calcular_area(self):
        return (self.diagonal_mayor * self.diagonal_menor) / 2
    def calcular_perimetro(self):
        return 4 * self.lado
class Trapecio:
    def __init__(self, base_mayor, base_menor, altura, lado1, lado2):
        self.base_mayor = base_mayor
        self.base_menor = base_menor
        self.altura = altura
        self.lado1 = lado1
        self.lado2 = lado2
    def calcular_area(self):
        return ((self.base_mayor + self.base_menor) * self.altura) / 2
    def calcular_perimetro(self):
        return self.base_mayor + self.base_menor + self.lado1 + self.lado2
figura1 = Circulo(2)
figura2 = Rectangulo(1, 2)
figura3 = Cuadrado(3)
figura4 = TrianguloRectangulo(3, 5)
figura5 = Rombo(8, 6, 5)
figura6 = Trapecio(10, 6, 4, 5, 5)
print("--- Circulo ---")
```



```
print(f"Área: {figura1.calcular_area():.2f}")
print(f"Perímetro: {figura1.calcular_perimetro():.2f}")
print("\n--- Rectángulo ---")
print(f"Área: {figura2.calcular_area()}")
print(f"Perímetro: {figura2.calcular_perimetro()}")
print("\n--- Cuadrado ---")
print(f"Área: {figura3.calcular_area()}")
print(f"Perímetro: {figura3.calcular_perimetro()}")
print("\n--- Triángulo Rectángulo ---")
print(f"Área: {figura4.calcular_area()}")
print(f"Hipotenusa: {figura4.calcular_hipotenusa():.2f}")
print(f"Perímetro: {figura4.calcular_perimetro():.2f}")
print(f"Tipo de triángulo: {figura4.tipo_triangulo()}")
print("\n--- Rombo ---")
print(f"Área: {figura5.calcular_area()}")
print(f"Perímetro: {figura5.calcular_perimetro()}")
print("\n--- Trapecio ---")
print(f"Área: {figura6.calcular_area()}")
print(f"Perímetro: {figura6.calcular_perimetro()}")
```

Diagrama de clases.



Punto 5.

Enunciado: Ejercicio 2.5 página 95

Se requiere un programa que modele una cuenta bancaria que posee los siguientes atributos:

- Nombres del titular.
- Apellidos del titular.
- Número de la cuenta bancaria.
- Tipo de cuenta: puede ser una cuenta de ahorros o una cuenta corriente.
- Saldo de la cuenta.

Se debe definir un constructor que inicialice los atributos de la clase. Cuando se crea una cuenta bancaria, su saldo inicial tiene un valor de cero. En una determinada cuenta bancaria se puede:

- Imprimir por pantalla los valores de los atributos de una cuenta bancaria. u Consultar el saldo de una cuenta bancaria.
- Consignar un determinado valor en la cuenta bancaria, actualizando el saldo correspondiente.
- Retirar un determinado valor de la cuenta bancaria, actualizando el saldo correspondiente. Es necesario tener en cuenta que no se puede realizar el retiro si el valor solicitado supera el saldo actual de la cuenta.

Ejercicios propuestos.

- Agregar a la clase Cuenta Bancaria, un atributo que represente el porcentaje de interés mensual aplicado a la cuenta.
- Agregar un método que calcule un nuevo saldo aplicando la tasa de interés correspondiente.

Código.

```
from enum import Enum
class TipoCuenta(Enum):
    AHORROS = "Ahorros"
    CORRIENTE = "Corriente"
class CuentaBancaria:
    def __init__(self, nombres_titular, apellidos_titular, numero_cuenta, tipo_cuenta,
porcentaje_interes=0.0,):
        self.nombres_titular = nombres_titular
        self.apellidos_titular = apellidos_titular
        self.numero_cuenta = numero_cuenta
        self.tipo_cuenta = tipo_cuenta
        self.saldo = 0.0
        self.porcentaje_interes = porcentaje_interes
    def imprimir_datos(self):
        print(f"Titular: {self.nombres_titular} {self.apellidos_titular}")
        print(f"Número de Cuenta: {self.numero_cuenta}")
```

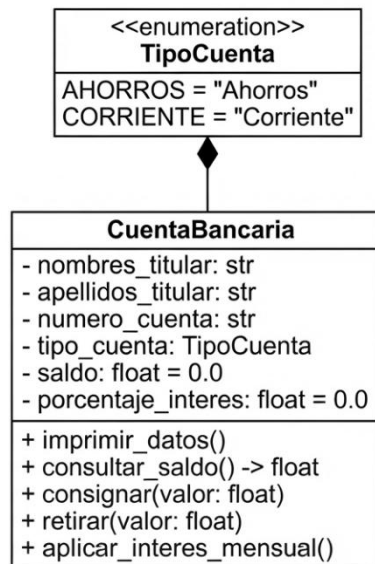


```
print(
    f"Tipo de Cuenta: {self.tipo_cuenta.value if self.tipo_cuenta else None}"
)
print(f"Saldo: ${self.saldo:.2f}")
print(f"Porcentaje Interés Mensual: {self.porcentaje_interes}%")
def consultar_saldo(self):
    print(f"El saldo actual de la cuenta es: ${self.saldo:.2f}")
    return self.saldo
def consignar(self, valor):
    if valor > 0:
        self.saldo += valor
        print(f"Se consignaron ${valor:.2f}. Nuevo saldo: ${self.saldo:.2f}")
    else:
        print("El valor a consignar debe ser mayor a cero.")
def retirar(self, valor):
    if valor > self.saldo:
        print("No se puede realizar el retiro. El valor supera el saldo actual.")
    elif valor <= 0:
        print("El valor a retirar debe ser mayor a cero.")
    else:
        self.saldo -= valor
        print(f"Se retiraron ${valor:.2f}. Nuevo saldo: ${self.saldo:.2f}")
def aplicar_interes_mensual(self):
    interes = self.saldo * (self.porcentaje_interes / 100)
    self.saldo += interes
    print(
        f"Interés aplicado (${interes:.2f}). Nuevo saldo tras interés: ${self.saldo:.2f}"
    )

cuenta1 = CuentaBancaria("Carlos", "Gómez", "1028394857", TipoCuenta.AHORROS, 1.5)
print("--- Datos Iniciales de la Cuenta ---")
cuenta1.imprimir_datos()
print("\n--- Operaciones ---")
cuenta1.consignar(500000)
cuenta1.retirar(100000)
cuenta1.retirar(600000)
cuenta1.aplicar_interes_mensual()
print("\n--- Estado Final de la Cuenta ---")
cuenta1.imprimir_datos()
```




Diagrama de clases.



URL Github:

<https://github.com/Santi0405/Actividades-POO>